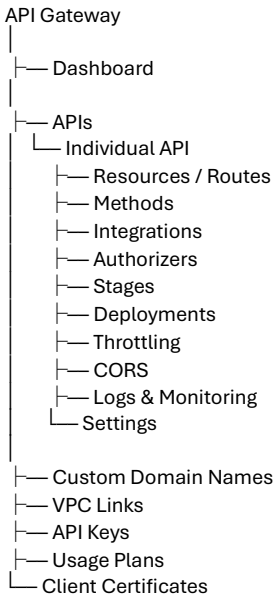
Amazon API Gateway — Console Dashboard Wise Important Interview Questions & Answers



Dashboard

1. What is Amazon API Gateway?

Answer:

Amazon API Gateway is a managed AWS service used to create and manage APIs.
It acts as a gateway between clients and backend services such as Lambda, EC2, or other applications.
It can handle security, throttling, monitoring, and API traffic management.

For example:

Suppose our mobile application needs customer data from a Lambda function.
We create an API in API Gateway and connect it to Lambda.
The mobile app calls the API instead of directly accessing Lambda.

Example:



Simple ga gurthupettuko:

API Gateway = Front door for APIs

APIs

2. What are APIs in API Gateway?

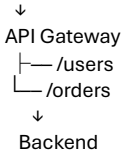
Answer:

An API is the main resource where we define how clients communicate with our backend application.
Inside an API, we configure routes or resources, HTTP methods, integrations, authorization, and stages.
API Gateway supports REST APIs, HTTP APIs, and WebSocket APIs.

For example:

Suppose our application needs /users and /orders endpoints.
We can create an API Gateway API and configure these endpoints to connect to our backend services.
Clients then use the API Gateway endpoint to access the application.

Example:



Simple ga gurthupettuko:

API = Main API configuration

Resources / Routes

3. What are Resources or Routes in API Gateway?

Answer:

Resources or routes define the API paths that clients can call.

A route normally contains a path such as /users or /orders/{id} and an HTTP method.
The route determines which backend integration should receive the request.
For example:
Suppose we create GET /users for retrieving users and POST /users for creating users.
API Gateway receives the request and sends it to the configured backend.
Different routes can connect to different backend services.
Example:
API Gateway


Simple ga gurthupettuko:
Route = API path + HTTP method

Methods
4. What are HTTP Methods in API Gateway?
Answer:
HTTP methods define what operation the client wants to perform through the API.
Common methods are GET, POST, PUT, PATCH, and DELETE.
We configure the required method and connect it to a backend integration.
For example:
GET /users can retrieve users, while POST /users can create a new user.
DELETE /users/10 can remove a specific user.
API Gateway routes each method to the correct backend.
Example:
GET → Read
POST → Create
PUT → Update
DELETE → Delete
Simple ga gurthupettuko:
HTTP Method = What operation client wants

Integrations
5. What is an Integration in API Gateway?
Answer:
An integration tells API Gateway where to send an incoming API request.
API Gateway can integrate with services such as Lambda, HTTP endpoints, AWS services, and VPC-based backends depending on the API type.
The integration connects the API layer with the actual application logic.
For example:
Suppose a client calls GET /users.
API Gateway can forward that request to a Lambda function.
Lambda processes the request and returns the response through API Gateway.
Example:
Client
↓
API Gateway
↓
Integration
↓
Lambda
↓
Response
Simple ga gurthupettuko:
Integration = Where API request goes

---

6. How does API Gateway work with Lambda?
Answer:
API Gateway can receive an HTTP request and invoke a Lambda function.
Lambda processes the request and returns the result to API Gateway.
API Gateway then sends the response back to the client.
For example:
Suppose we create GET /products in API Gateway.
The route is connected to a Lambda function that reads products from DynamoDB.
The client receives the product data through API Gateway.
Example:
Client
↓
API Gateway

↓
Lambda
↓
DynamoDB
↓
Lambda
↓
API Gateway
↓
Client

Simple ga gurthupettuko:
API Gateway + Lambda = Serverless API

Authorizers

7. What is an Authorizer in API Gateway?

Answer:
An authorizer controls whether a client is allowed to access an API.
It checks the identity or credentials provided with the request before allowing access to protected routes.
API Gateway supports authorization mechanisms such as IAM and supported authorizer options.

For example:
Suppose /admin should only be accessible to authenticated users.
The client sends authentication information with the request.
The authorizer validates it before API Gateway allows the request to reach the backend.

Example:
Client
↓
API Gateway
↓
Authorizer
├── Valid → Backend
└── Invalid → Deny

Simple ga gurthupettuko:
Authorizer = Check who can access API

Stages

8. What is a Stage in API Gateway?

Answer:
A stage is a logical environment where an API deployment is available.
Common stages can be dev, test, and prod.
Each stage can have its own configuration such as throttling, logging, and variables.

For example:
Suppose developers test a new API in dev while customers use prod.
We can deploy the API separately to these stages.
This allows us to manage development and production independently.

Example:
API
│
├── dev
├── test
└── prod

Simple ga gurthupettuko:
Stage = Environment for deployed API

Deployments

9. What is an API Gateway Deployment?

Answer:
A deployment makes a configured version of an API available to a stage.
After changing routes, methods, or integrations, the changes may need to be deployed to the required stage.
Without the required deployment, clients may not receive the latest API configuration.

For example:
Suppose we add a new /orders route to our API.
After configuring it, we deploy the API to the prod stage.
The production stage can then serve the updated API configuration.

Example:
API Changes
↓
Deployment
↓
prod Stage
↓
Clients

Simple ga gurthupettuko:
Deployment = Publish API changes to a stage

Throttling
10. What is API Gateway Throttling?
Answer:
Throttling controls how many API requests can be processed during a period of time.
It helps protect backend services from excessive traffic and sudden request spikes.
We can configure throttling at appropriate API or usage levels depending on the API type and configuration.
For example:
Suppose our backend can safely handle only a certain number of requests per second.
We configure throttling so excessive requests are limited instead of overwhelming the backend.
This helps protect application availability.
Example:
Many Requests
↓
API Gateway
↓
Throttling
├── Allowed → Backend
└── Excess → Throttled
Simple ga gurthupettuko:
Throttling = Control request rate

CORS
11. What is CORS in API Gateway?
Answer:
CORS stands for Cross-Origin Resource Sharing.
It controls whether a browser-based application from one origin can call an API hosted on another origin.
We configure allowed origins, methods, and headers according to the application requirement.
For example:
Suppose our frontend runs on www.myapp.com and the API is hosted through API Gateway on another domain.
The browser may block the request unless the API provides the required CORS configuration.
We configure CORS to allow the required frontend origin.
Example:
Frontend
www.myapp.com
↓
Browser
↓
API Gateway
↓
CORS Rules
↓
Backend
Simple ga gurthupettuko:
CORS = Allow browser requests from another origin

Logs & Monitoring
12. How do you monitor API Gateway?
Answer:
API Gateway can send monitoring information to Amazon CloudWatch.
We can monitor metrics such as request count, latency, errors, and throttled requests.
Logs can also help us troubleshoot API requests and backend problems.
For example:
Suppose users report that an API is responding slowly.
We can check API Gateway CloudWatch metrics and logs to identify latency or error patterns.
This helps us troubleshoot the API.
Example:
API Gateway
↓
CloudWatch
├── Requests
├── Latency
├── Errors
└── Throttling
Simple ga gurthupettuko:
Monitoring = API metrics + logs in CloudWatch

Custom Domain Names
13. What is a Custom Domain Name in API Gateway?

Answer:
A custom domain allows clients to access an API using our own domain instead of the default API Gateway URL.
We configure the domain and associate it with the required API and stage.
HTTPS certificates are commonly used to secure the custom domain.
For example:
Instead of giving users a long API Gateway URL, we can use api.myapp.com.
Route 53 can point the domain to API Gateway.
Users can then call the API using our custom domain.
Example:
api.myapp.com
↓
Route 53
↓
API Gateway
↓
Backend
Simple ga gurthupettuko:
Custom Domain = Your own domain for API

VPC Links
14. What is a VPC Link in API Gateway?
Answer:
A VPC Link allows API Gateway to access private backend resources inside a VPC.
It is useful when the backend should not be directly exposed to the public internet.
Depending on the API Gateway API type, VPC Link can connect the API to private load balancers or supported VPC resources.
For example:
Suppose our backend application is running privately behind an internal Load Balancer.
API Gateway can use a VPC Link to reach that private backend.
The backend does not need to have a public endpoint.
Example:
Internet Client
↓
API Gateway
↓
VPC Link
↓
Internal Load Balancer
↓
Private EC2
Simple ga gurthupettuko:
VPC Link = API Gateway → Private VPC backend

API Keys
15. What is an API Key?
Answer:
An API key is a value that can be used to identify and control API clients.
API keys can be associated with usage plans to control request quotas and throttling.
They should not be treated as the primary authentication mechanism for sensitive APIs.
For example:
Suppose we provide an API to different application clients.
We can issue different API keys and associate them with usage plans.
This allows us to track and control their API usage.
Example:
Client
↓
API Key
↓
API Gateway
↓
Usage Plan
↓
API
Simple ga gurthupettuko:
API Key = Identify/control API usage

Usage Plans
16. What is a Usage Plan?
Answer:
A usage plan defines how API clients can use an API when API keys are used.
It can include throttling and quota settings for controlling API consumption.
This is useful when different clients need different usage limits.

For example:
Suppose Client A should make more requests than Client B.
We can create appropriate usage plans and associate the clients' API keys with them.
API Gateway can then enforce the configured limits.
Example:
Usage Plan
 ├— Throttling
 ├— Quota
 └— API Key
  ↓
  Client
Simple ga gurthupettuko:
Usage Plan = Control how much a client can use API

Most Important Interview Scenario
17. Explain a real-time API Gateway architecture.
Answer:
A client sends an HTTP request to an API Gateway endpoint.
API Gateway authenticates and controls the request, then forwards it to the backend integration.
The backend processes the request and API Gateway returns the response to the client.
For example:
Suppose our shopping application has a GET /products API.
The request reaches API Gateway, which invokes Lambda, and Lambda reads product data from DynamoDB.
The response travels back through API Gateway to the application.
Example:
User / Mobile App
  ↓
 API Gateway
  ↓
 Authorizer
  ↓
  Lambda
  ↓
 DynamoDB
  ↓
  Lambda
  ↓
 API Gateway
  ↓
 User / App
Simple ga gurthupettuko:
API Gateway = Client → Secure API → Backend → Response